

AWS Certified Generative AI Developer — Professional

Domain 1, Task 1.1 — Analyse requirements and design GenAI solutions · Revision notes

The Use Case

- Multinational bank — 15 countries, 8 languages
- **Requirements:** multilingual support, domain-specific financial knowledge, CRM integration
- **Constraints:** financial regulation, data privacy, **data residency**, cross-region consistency
- **Target:** 70% reduction in response time

Request Flow, In Order

1. DNS

- App resolves the hostname to **2 static IPs**
- The same 2 IPs are returned to everyone worldwide

2. Anycast

- Same IPs announced from 200+ edge locations via **BGP**
- No lookup, no geolocation, no decision — each router independently picks its shortest path
- "Shortest" means network hops, not kilometres — routing can occasionally be counter-intuitive
- **Global Accelerator plays no role at this stage**

3. AWS Global Accelerator

- **Enters only at the edge**, after the packet has already arrived there
- Selects the nearest **healthy** regional endpoint permitted by configuration
- Forwards over the **AWS backbone** rather than the public internet — this is the actual latency benefit
- 2 static IPs = redundancy across separate network zones — **not** one per region
- IPs are permanent → allowlist-friendly for partner banks and corporate firewalls
- **Config hierarchy:** Accelerator → Listener → Endpoint Group (one per region)

No native "EU users stay in EU" setting. Enforce data residency via traffic dial 0%, or by creating separate accelerators per jurisdiction.

- Single-region deployment → GA gives backbone benefit only, no failover value. **CloudFront is often the better choice** if content is cacheable.

4. AWS AppSync

- GraphQL API layer — entry point, auth enforcement, routing to backend
- **Subscriptions** = built-in WebSocket → token streaming. This is the strongest justification for choosing it here.
- Field-level authorization declared in the schema, enforced before resolvers run
- One round trip for related data; direct resolvers to DynamoDB or Bedrock possible
- Timeout ≈ 30s — **the same as API Gateway**, so not a differentiator
- Escapes the timeout by going async: mutation returns a `conversationId` in milliseconds, results pushed over the subscription
- **Setup order:** API → Schema → Data Source → Resolver
- Schema defines *what* is possible; Resolver defines *how* it happens

5. Amazon Cognito

- User Pool created first; App Client for the mobile application
- Token issued at **login**, not per request
- AppSync validates the JWT signature **locally** against cached JWKS — no network call to Cognito
- Claims carried: `sub`, `cognito:groups`, `custom:jurisdiction`
- Jurisdiction comes from the signed token → **cannot be spoofed by the client**
- A deleted user's token stays valid until expiry (~1 hr) — inherent to JWT, not an AppSync flaw

6. AWS Lambda

- Stateless orchestration; 15-minute ceiling; cold starts
- **Sequence:** retrieve chunks → fetch prompt template → call model → emit event

7. Bedrock Knowledge Bases — Retrieval

The model never searches anything. Lambda retrieves first, then passes the chunks into the prompt. The reference diagram draws these arrows backwards.

- Query → embedding → vector search (cosine similarity) → metadata filter → top-k chunks
- Multilingual embeddings place the French question near the English document → cross-language retrieval works
- **Metadata filter applied before ranking** (e.g. `jurisdiction = FR`) — this is the actual compliance enforcement mechanism
- APIs: `Retrieve` (you generate) vs `RetrieveAndGenerate` (managed). The order never changes.
- Optional refinements: hybrid search, re-ranking

8. Bedrock Prompt Management

- Versioned templates with `{{context}}` and `{{question}}` slots
- Compliance ships v8 with new disclaimer wording → **zero code deployment**
- Rollback is a version pointer change

9. Amazon Bedrock → Claude

- Prompt = system instructions + retrieved chunks + user question
- **Model supplies language and reasoning; retrieval supplies all facts**
- `InvokeModelWithResponseStream` for token streaming
- Bedrock concepts: unified API across providers, on-demand vs provisioned throughput, cross-region inference profiles, per-model access grants

10. Response streams back

- Lambda → AppSync → WebSocket → user
- Chunk size is determined by the **model's** token generation, not by AppSync

11. Amazon EventBridge

- `PutEvents` — fire-and-forget, **after** the user already has their answer
- Rules route by content: CRM Lambda, low-confidence review queue, analytics
- CRM outage → retries with backoff, then DLQ. Customer unaffected.
- A synchronous CRM call would turn a CRM outage into a **customer-facing outage**
- Adding a new consumer = adding a new rule; producer code untouched

Ingestion — Offline, Before Any Query

1. **Connect data source** — S3, SharePoint, Confluence
2. **Extract text** — formatting, images and layout discarded
3. **Chunking** — ~300-500 words, ~10% overlap
 - Strategies: fixed size / semantic / hierarchical
4. **Embed** — each chunk gets its own vector, one at a time
 - **The same embedding model must be used at query time**, or the vectors won't match
5. **Store** — chunk text + vector + metadata together
 - Metadata must be added here; adding it later requires full re-ingestion

Sync: manual, scheduled, or event-driven (S3 change → EventBridge → sync). Incremental sync reprocesses only changed documents.

Exam-relevant: when RAG performs badly, chunking is the cause roughly 80% of the time — not the model.

Vector Stores

Store	When to use
OpenSearch Serverless	Lowest latency; real-time chat
S3 Vectors	Large datasets, cost-optimised; ~100 ms warm, 90%+ recall, sub-second acceptable
Aurora pgvector	Already running PostgreSQL
Neptune Analytics	Graph-enhanced RAG

Critical: SharePoint, Confluence, or Salesforce data sources support **only OpenSearch Serverless**. This rules out S3 Vectors for the bank scenario in the reference architecture.

- S3 Vectors limits: 1 KB metadata and 35 metadata keys per vector
- **Knowledge Bases does not store vectors.** The vector store is a separate provisioned, billed component — and it is missing from the reference diagram.

Guardrails — Missing From The Reference Diagram

Applied twice: once on input, once on output.

Stage	Screens for
Input	Prompt injection, PII, denied topics
Output	Denied topics (e.g. no investment advice), PII masking, contextual grounding

Contextual grounding asks: did the answer actually follow from the retrieved chunks? Two scores — grounding and relevance — each with a configurable threshold. This is the direct control for hallucination.

Why it sits outside the model: applies across all models, independently versioned and auditable, and cannot be bypassed by changing the prompt.

Monitoring — Three Distinct Jobs

Service	Question it answers
CloudWatch	How many? How slow? — metrics and alarms
X-Ray	<i>Where</i> did the time go? — per-hop trace of a single request
SageMaker Pipelines	Are answers still correct? — offline, scheduled

- Bedrock metrics: `InvocationLatency` , `InputTokenCount` , `OutputTokenCount` , `InvocationThrottles`
- Model invocation logging writes full prompts and completions to S3 — required for audit, and itself a privacy consideration
- SageMaker Pipelines replays a golden dataset in all languages, scoring accuracy, bias and disclaimer presence. It catches silent regression caused by a prompt change — something CloudWatch cannot see, because latency and error rates stay normal.
- Bedrock **Model Evaluation** overlaps at smaller scale; MLflow for experiment tracking

Service Discriminations

Choice	Pick when
Global Accelerator	Multi-region failover, non-cacheable traffic, static IPs required
Route 53 latency routing	DNS-level routing acceptable, TTL delay tolerable
CloudFront	Cacheable content
AppSync	GraphQL, subscriptions, mobile over-fetching, multiple client shapes, offline sync
API Gateway	REST, usage plans, API keys, per-customer throttling tiers
Lambda Function URL	Response streaming — API Gateway cannot stream, it buffers
EventBridge	Content-based filtering, many targets, SaaS integration
SNS	Simple fan-out, no filtering needed
SQS	Point-to-point queue, one consumer group
Bedrock Flows	Deterministic multi-step graph, auditable — fits regulated domains
Bedrock Agents	Model chooses its own tool sequence; flexible, harder to audit
RAG	Facts change often, citations needed, per-tenant filtering needed
Fine-tuning	Need style, format or domain behaviour — <i>not</i> fresh facts

Errors In The Reference Diagram — Worth Remembering

- **Arrows from Claude → Knowledge Bases / Prompt Management / Flows are backwards.** The application calls the KB, receives chunks, *then* calls the model.
- **No vector store shown** — it is required, provisioned and billed separately.
- **No Guardrails** — the most serious omission for a regulated workload.
- **Multi-region vs a single Knowledge Base is contradictory** — data residency requires a KB and vector store per jurisdiction.
- **No caching** — prompt caching and semantic caching are the standard cost levers at 3× volume.

Deployment And Hosting Strategies

Four factors drive the decision: **model size, latency requirement, cost constraints, scaling pattern.**

Option	Choose when
Bedrock On-Demand	Uneven traffic, pay per token, no commitment
Bedrock Provisioned Throughput	Steady high volume; guaranteed capacity, no throttling. More expensive but predictable.
Bedrock Batch	Results not needed in real time — roughly 50% cheaper
SageMaker real-time endpoint	Your own model, always on, low latency required
SageMaker Serverless Inference	Your own model, intermittent traffic, cold starts acceptable
SageMaker Asynchronous Inference	Large payloads or long-running inference
Amazon EC2	Full control, self-managed GPU infrastructure

Cross-region inference profiles distribute Bedrock inference across regions, reducing throttling under load.

Amazon Q vs Bedrock vs SageMaker AI

Three different levels of abstraction, not three competitors.

Amazon Q	-> ready-made application (no code)
Bedrock	-> managed API (some code)
SageMaker AI	-> your model, your training (full code)

Service	What it is
Amazon Q Business	Enterprise search and chat assistant with 40+ data connectors, aimed at business users. You connect data; you don't build the app.
Amazon Q Developer	Coding assistant
Amazon Bedrock	Unified API across FM providers for building your own application
SageMaker AI	Train, fine-tune and host your own models with full control

Rule: off-the-shelf will do → Q. Building a custom application → Bedrock. Training the model yourself → SageMaker.

Generative AI Application Builder on AWS — an AWS Solution deployed via CloudFormation, with pre-built LLM connectors through Bedrock, a UI and RAG support. Designed for **rapid PoC work without deep AI experience.**

RAG vs Flows vs Agents

Pattern	Who decides the sequence
RAG	You — retrieve, then generate. Fixed.
Flows	You — a graph defined in advance: prompt nodes, KB nodes, Lambda nodes, conditions
Agents	The model — chooses which tool to call and when

How Agents work:

- **Action groups** — Lambda functions the agent may call, described by an OpenAPI schema
- **ReAct loop** — reason, invoke a tool, observe the result, reason again
- **Memory** across sessions

Agents are non-deterministic — the same question can take a different path each time. Flexible, but hard to audit. This is why the regulated banking scenario uses **Flows rather than Agents.**

Hybrid: an Agent can have RAG as one of its tools.

Event-Driven Integration Patterns

Service	Choose when
SQS	Queue with one consumer group; buffering and natural rate limiting
SQS FIFO	Strict ordering required
SNS	Simple fan-out, no content filtering
EventBridge	Content-based routing, many targets, SaaS integrations, schema registry
Kinesis Data Streams	Ordered streaming with replay
Amazon MSK (Kafka)	Existing Kafka estate, or very high throughput
Webhooks	Pushing events out to external systems

GenAI-specific consideration: FM calls take seconds, not milliseconds. **Asynchronous integration should be the default** — queue the request, deliver the result via callback or subscription, rather than holding a synchronous connection open.

Well-Architected Framework + Generative AI Lens

Six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

The **Generative AI Lens** applies GenAI-specific questions to each:

Pillar	GenAI-specific concerns
Operational Excellence	Prompt versioning, model evaluation pipelines, GenAIOps
Security	Prompt injection, PII handling, Guardrails, per-model access control
Reliability	Throttling, fallback models, graceful degradation
Performance Efficiency	Model selection, caching, response streaming
Cost Optimization	Token optimisation, right-sizing the model, batch inference, caching
Sustainability	Using the smallest model that does the job

Reviews are run through the **AWS Well-Architected Tool** with the Lens selected.

GenAIOps And Governance

- **SageMaker Pipelines** — standardised, repeatable workflows for experimentation, training and deployment
- **MLflow** (managed within SageMaker) — experiment tracking: which combination of prompt, model and parameters performed best
- **Governance:** prompt versioning, model versioning, IAM access controls, KMS encryption
- **Standardised evaluation:** consistent metrics and test procedures for performance, bias and business alignment across every deployment

Reusable Infrastructure Components

Service	Role in a GenAI application
EC2	Running applications, self-managed model hosting
S3	Data, documents and generated outputs
Lambda	Serverless, event-driven orchestration
CloudWatch	Monitoring and alarms
CloudFormation	Infrastructure as code, deployed consistently across regions

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.